DOCUMENTACIÓN DEL SISTEMA

Proyecto AW
Guía técnica de los proyectos y sus métodos

Backend administrativo, API y aplicación web móvil para operadores

Edición de referencia · 19 de octubre de 2026

Qué explica esta guía

Este documento sirve para entender dónde vive cada responsabilidad, cómo fluye una guía desde su creación hasta el movimiento de stock y cómo ampliar el sistema sin duplicar lógica. La conclusión principal es sencilla: ProjectAW es la fuente de verdad para reglas, persistencia y seguridad; ProjectAW.PWA sirve la interfaz del operador y conversa con esa API por HTTP/JSON, sin implementar controladores de negocio propios.

> **Alcance** Se describe la implementación actual observada en el repositorio, con ejemplos adaptados para reutilizar sus patrones. Los ejemplos son orientativos: antes de pegarlos en producción se deben ajustar al caso y validar permisos, contratos y migraciones.

Mapa rápido

Parte	Responsabilidad
ProjectAW	Aplicación ASP.NET Core MVC y API. Reglas de guías, autenticación, acceso a SQL Server, lecturas, stock, movimientos y vistas de administración.
ProjectAW.PWA	Aplicación ASP.NET Core mínima que publica archivos estáticos HTML, CSS y JavaScript para operadores. No contiene DbContext ni controladores API propios.
Navegador del operador	Ejecuta módulos ES de JavaScript. Usa fetch contra la URL de ProjectAW; envía la cookie de sesión con credentials: include.
SQL Server	Guarda catálogos, guías, detalle, lecturas, existencias y movimientos; EF Core materializa consultas y aplica cambios.

```
Operador → navegador/PWA → HTTPS fetch + cookie → ProjectAW API → servicios/repositorios → EF Core → SQL Server
```

1. Los dos proyectos y cómo se comunican

ProjectAW como backend y panel administrativo

ProjectAW es el proyecto con acceso a la base de datos y con controladores MVC/API. El panel de administración crea y consulta información; las APIs publican contratos JSON para el panel y para otros clientes autorizados. Las reglas no deben depender de que el navegador las aplique: se validan de nuevo en servicios y repositorios.

- Tecnología comprobada en el proyecto: ASP.NET Core MVC, Entity Framework Core 10, SQL Server y Swagger disponible en desarrollo.
- El punto de entrada Program.cs registra controladores/vistas, CORS, autenticación por cookies, rate limiter para inicio de sesión, servicios, repositorios y AppDbContext.
- AppDbContext agrega configuraciones por ensamblado y declara secuencias SQL para los números de guía.

ProjectAW.PWA como cliente web móvil

ProjectAW.PWA se publica en su propio host (en desarrollo, https://localhost:7144). Program.cs solo configura HTTPS, archivos por defecto y StaticFiles. Al abrir la raíz sirve el login estático. Las pantallas operativas son archivos HTML y cada archivo carga un módulo JavaScript.

- No tiene referencia al proyecto ProjectAW, no registra un DbContext y no contiene endpoints de negocio propios.
- El JavaScript llama a fetch desde el navegador hacia https://localhost:7054, según wwwroot/config/api-config.js.
- Que haya dos hosts no significa que la PWA “llame internamente” a un controlador suyo: las llamadas cruzan la red HTTP y llegan al backend API.
- El host de la PWA sigue siendo un servidor ASP.NET, pero su función es servir recursos estáticos. No es una segunda API de negocio.

Ejemplo del cliente HTTP

```
// api-client.js
const response = await fetch(`${apiBaseUrl}/api/guias?idEstadoGuia=1`, {
  method: "GET",
  credentials: "include",
  headers: { Accept: "application/json, application/problem+json" }
});
const data = await response.json();
```

El navegador obtiene el JSON desde la API y la interfaz crea las tarjetas. Si el usuario no tiene sesión o rol apropiado, el backend responde 401/403 y la PWA redirige o muestra un mensaje. CORS permite que el navegador lea una respuesta entre orígenes; por sí solo no autentica ni concede permisos.

Desarrollo local

Proceso	Perfil / URL de desarrollo
ProjectAW API	HTTPS: https://localhost:7054 (HTTP: http://localhost:5016).
ProjectAW.PWA	HTTPS: https://localhost:7144 (HTTP: http://localhost:5144).
Origen permitido	En el perfil HTTPS del backend: Cors__AllowedOrigins__0=https://localhost:7144.

Inicio conjunto En Visual Studio, configura ProjectAW y ProjectAW.PWA como proyectos de inicio múltiples. La URL de la API de producción se configura en api-config.js; nunca se debe dejar vacía al desplegar.

2. Capas del backend y por qué se usan

Capa	Qué hace	Qué evitar
Controller	Recibe HTTP, lee ruta/query/body, llama al servicio y devuelve código/JSON HTTP.	Reglas complejas, SQL directo y actualización de múltiples entidades.
DTO	Define el contrato de entrada o respuesta y valida campos del request.	Enviar entidades EF tal cual; acoplar la base al cliente.
Service	Coordina reglas del negocio, permisos de operación y conversión a DTO.	Confiar únicamente en validaciones HTML/JavaScript.
Repository	Consulta y actualiza EF Core; concentra filtros, Includes, transacciones y SaveChanges.	Duplicar consultas SQL en varios controladores.
Entity + Configuration	Representa persistencia y restricciones de esquema/relación.	Usar la entidad como formato público por comodidad.
DbContext	Unidad de trabajo de EF y puente a SQL Server.	Instanciar contextos manualmente en cada request.

Inyección de dependencias

Program.cs registra implementaciones por interfaz con AddScoped. Scoped crea una instancia por solicitud HTTP, que encaja con el ciclo de vida de AppDbContext. Un servicio depende de una interfaz de repositorio y el controller de una interfaz de servicio: eso reduce el acoplamiento y permite cambiar o simular una implementación sin reescribir el consumidor.

```
builder.Services.AddScoped<IGuiaRepository, GuiaRepository>();
builder.Services.AddScoped<IGuiaService, GuiaService>();

public GuiasApiController(IGuiaService service) => _service = service;
```

Asincronía y CancellationToken

Los métodos de base de datos usan Task/async/await (`ToListAsync`, `SaveChangesAsync`) para no bloquear un hilo ASP.NET mientras SQL trabaja. Los controladores reciben CancellationToken y lo pasan al servicio/repositorio; si el cliente cancela o la solicitud vence, la consulta puede detenerse. Se evita `.Result` o `.Wait()`, que bloquean hilos y pueden empeorar la concurrencia.

```
public async Task<IActionResult> Listar(CancellationToken ct)
    => Ok(await _service.ListarAsync(idEstadoGuia: 1, ct));
```

Dónde se encuentran las reglas de guías

- GuiasApiController: rutas HTTP `api/guias` y respuesta HTTP (GET, POST, PUT, cancelar).
- GuiaService: valida operación, relaciones, productos, cantidades y estado antes de crear/editar/cancelar.
- GuiaRepository: hace consultas EF, secuencias y persistencia.
- GuiaResponse y requests: separan el contrato HTTP de `CabeceraGuia` y `DetalleGuia`.

Módulos funcionales

Módulo	Responsabilidad que cubre
Guías	Cabecera y detalles de entrada, salida y traslado; creación, consulta, edición y cancelación según el estado.
Maestros	Productos, categorías, almacenes, ubicaciones, clientes, proveedores, unidades de medida, roles y tipos de documento.
Seguridad	Usuarios, roles, login, logout, claims y sesiones separadas para administrador/operador.
Stocks	Saldo por almacén y ubicación, historial de movimientos y aplicación de lecturas de guías.

Módulo	Responsabilidad que cubre
Operaciones PWA	Listados por operador, detalle, captura PDA/cámara, historial, avance y confirmación de cierre.

Los maestros suelen repetir el mismo patrón CRUD: controller API con rutas GET/POST/PUT, servicio con reglas y repositorio con EF Core. Las vistas MVC para administración y sus controladores web sirven páginas; las APIs devuelven datos. `ModuleViewLocationExpander` permite organizar vistas por módulo en vez de concentrarlas en una carpeta global.

GuiaService métodos principales

Método	Uso y motivo
CrearAsync	Valida cabecera, líneas y referencias; genera número; asigna estado Pendiente y usuario creador; guarda y devuelve DTO.
ListarAsync	Aplica filtros opcionales por operación/estado y proyecta cada guía a respuesta.
ObtenerPorIdAsync	Centraliza carga o error 404 y convierte a respuesta.
ActualizarAsync	Solo permite editar Pendiente. Revalida y sustituye detalles para guardar una versión coherente.
CancelarAsync	Solo cancela Pendiente; registra usuario/fecha de modificación y cambia el estado.

Regla importante Las validaciones del servicio son necesarias aunque existan DataAnnotations y controles de pantalla. La API se puede llamar directamente, así que el servidor debe rechazar estados, relaciones o cantidades inválidas.

3. API disponible y contratos relevantes

Los controladores API se mapean con autorización global en Program.cs (`MapControllers().RequireAuthorization()`). El login es una excepción explícita con AllowAnonymous. Las rutas de datos de PWA reciben cookies y reglas CORS delimitadas; la API sigue comprobando autenticación y autorización.

Área / método	Ruta de ejemplo	Uso
Sesión · POST	/api/auth/login	Valida credenciales y emite cookie de administrador u operador según rol y origen.
Sesión · POST	/api/auth/logout	Cierra la cookie del cliente correspondiente.
Guías · GET	/api/guias?idTipoOperacionGuia=1&idEstadoGuia=1	Lista guías por operación y estado.
Guías · GET	/api/guias/{id}	Consulta el detalle para mostrar y confirmar una guía.
Guías · POST/PUT	/api/guias y /api/guias/{id}	Crea y actualiza guías desde el panel que corresponda.
Catálogos · GET	/api/ProductoApi, /api/AlmacenApi, /api/ProveedorApi, /api/ClienteApi	Carga nombres/datos asociados a IDs.
Ubicaciones · GET	/api/UbicacionApi?idAlmacen=4&soloActivas=true	Devuelve ubicaciones válidas del almacén.
Operador · GET	/api/simulacion/guias/mias?idEstadoGuia=2	Lista únicamente las guías del operador autenticado.
Operador · GET	/api/simulacion/guias/mias/{idGuia}	Detalle sujeto a pertenencia a la guía.
Operador · POST	/api/simulacion/guias/{idGuia}/tomar	Asigna una guía pendiente al operador actual.
Operador · POST/GET	/api/simulacion/guias/{idGuia}/lecturas	Persiste una captura o consulta historial.
Operador · POST	/api/simulacion/guias/{idGuia}/finalizar	Aplica movimientos, existencias y estado en una transacción.
Inventario · GET	/api/stocks?idAlmacen=4&idProducto=23	Consulta saldos por almacén/producto.
Movimientos · GET	/api/movimientos-stock?idGuia=9	Consulta trazabilidad histórica con filtros opcionales.

Estados y operaciones

ID	Operación	ID	Estado de guía
1	Entrada / Recepción	1	Pendiente
2	Salida / Despacho	2	En proceso
3	Traslado	3	Atendida
		4	Cancelada

Las constantes están en `TipoOperacionGuiaIds` y `EstadoGuiasIds`. En UI pueden traducirse a Recepción/Despacho/Traslado y Pendiente/En proceso/Terminada. Conviene usar las constantes en C# y mantener un mapeo explícito en JavaScript para no dispersar números mágicos.

Respuestas y errores

La API devuelve JSON y, cuando falla, Problem Details con `status`, `title`, `detail`, la propiedad española `mensaje` y `traceId`. ApiExceptionHandler convierte errores conocidos a respuestas seguras: regla inválida 400, faltante 404, conflicto de negocio/datos 409 y error inesperado 500. El detalle interno del 500 se registra en el servidor, no se revela al cliente.

HTTP	Interpretación para el cliente

HTTP	Interpretación para el cliente
400	El request o las reglas del negocio no se cumplen; mostrar mensaje entendible.
401	No hay sesión válida; volver al acceso.
403	La cuenta o el rol no tiene permiso para esa aplicación/acción.
404	El ID ya no existe o no pertenece a la vista del operador.
409	Conflicto de estado, stock insuficiente o duplicidad.
429	El limitador rechazó intentos de login demasiado frecuentes.
500	Fallo no previsto; informar el traceId si soporte necesita correlacionarlo.

4. Flujo real de una guía de operador

De Pendiente a En proceso

1. La PWA consulta tres listados de guías con `getGuides(tipo)` y filtra estado 1.
2. En la pantalla de operación, al tocar una tarjeta, `selectGuide` vuelve a consultar `GET /api/guias/{id}` para leer estado/código/fecha actuales.
3. Se muestra el diálogo de confirmación. Cancelar no modifica el backend.
4. Al aceptar, la PWA hace POST `/api/simulacion/guias/{id}/tomar`. El repositorio abre una transacción serializable, revalida que siga Pendiente, asigna usuario operador autenticado y cambia a En proceso.
5. La guía aparece en Mis guías del operador actual. El filtro del servidor usa el ID de usuario obtenido de claims, no un ID suministrado por el navegador.

Por qué consulta dos veces El listado puede quedar desactualizado mientras otro operador toma la guía. El GET de detalle actualiza la confirmación y el backend vuelve a validar dentro de la transacción antes de asignarla.

Capturar lecturas sin aplicar aún el stock

La PWA carga el detalle asignado, catálogo, ubicaciones e historial. Escanear PDA (actúa como teclado) o cámara identifica el producto de la guía. Enviar registro crea una fila `LecturaGuia`: producto, código validado, cantidad, operador, lote/serie, observación, ubicación y fecha UTC. Cada captura queda persistida y recuperable, pero todavía no cambia el inventario.

```
{
  "idProducto": 23,
  "cantidad": 4,
  "codigoLeido": "SKU-023",
  "lote": "L-2026-10",
  "serie": null,
  "observacion": "Caja revisada",
  "idUbicacionOrigen": null,
  "idUbicacionDestino": 7
}
```

La lista de historial permite verificar capturas anteriores. El avance suma cantidades por producto. Las ubicaciones se validan por tipo de operación: entrada requiere destino; salida requiere origen; traslado requiere ambas.

Finalizar y aplicar stock

6. El operador revisa esperado, leído y diferencia en el cuadro de confirmación.
7. La API verifica pertenencia, estado En proceso, producto por producto, cantidad entera y ubicaciones activas del almacén correcto.
8. El repositorio en transacción serializable aplica entradas/salidas/traslados a `StockUbicacion`, sincroniza `StockAlmacen` y agrega un `MovimientoStock` por producto.
9. Solo después de guardar todo cambia la guía a Atendida. Si falla una validación o el stock de origen no alcanza, se revierte la transacción completa.

La diferencia no es la cantidad a mover El movimiento usa el total leído; `CantidadDiferenciada` guarda leído menos esperado como dato de auditoría. La UI informa la diferencia antes de finalizar, pero el cierre no exige igualdad exacta.

Ejemplo de secuencia desde PWA

```
// Funciones centralizadas en modules/operaciones/guias.js
const guia = await getMyGuideById(9);
const lectura = await registerGuideReading(9, request);
const historial = await getGuideReadingHistory(9);
await finalizeGuideReadings(9);
```

La secuencia completa también depende de cookie vigente, permiso de Operador, asignación del ID de usuario, estado y reglas del servidor. El ejemplo no permite omitir esos controles.

5. Movimiento de stock y consultas

MovimientoStockService

Orquesta el caso de uso. En operaciones de operador comprueba que la cuenta esté activa y tenga rol Operador; luego delega persistencia. Para reportes convierte entidades a `MovimientoStockDto`, incluyendo códigos/nombres de producto, almacén, guía, ubicaciones y usuario.

Método	Responsabilidad
TomarGuiaAsync	Valida operador y delega asignación atómica.
ListarMisGuiasAsync	Admite solo estados En proceso o Atendida y limita por usuario autenticado.
ObtenerMiGuiaAsync	Consulta una guía del operador; evita que un ID arbitrario dé acceso a otras guías.
RegistrarLecturaAsync	Autoriza, valida la lectura a través del repositorio y la devuelve como DTO.
ListarLecturasAsync	Devuelve historial ordenado para una guía del operador.
FinalizarLecturaAsync	Comprueba forma de guía/operación/cantidades y ordena aplicar movimiento.
ListarAsync	Consulta movimientos con filtros de guía, almacén u operación; normalmente se usa en reportes administrativos.

MovimientoStockRepository

- Usa `AsNoTracking()` en lecturas que no modificarán entidades: menos carga del ChangeTracker.
- Usa `Include` cuando el DTO necesita propiedades relacionadas, por ejemplo el código de producto o ubicación.
- Envuelve tomar/finalizar en `BeginTransactionAsync(IsolationLevel.Serializable)`. Esa operación es crítica porque dos procesos concurrentes podrían leer el mismo estado/stock.
- `ValidarGuiaEnProceso` asegura estado y responsable; `ValidarUbicacionesAsync` verifica que la ubicación pertenezca al almacén y continúe activa.
- `AjustarStockUbicacionAsync` impide saldos negativos, duplicados o desbordamiento de int.
- `SincronizarStockAlmacenAsync` calcula el total desde saldos por ubicación. Es un método interno: solo se necesita al aplicar movimientos, no en una lectura normal de stock.

Qué diferencia a los dos saldos

Entidad	Grano de dato	Uso
StockUbicacion	Producto + ubicación	Saldo operativo detallado para picking/recepción; el par es único.
StockAlmacen	Producto + almacén	Resumen derivado de ubicaciones; se sincroniza al finalizar el movimiento.
MovimientoStock	Guía + producto por cierre	Libro de movimientos y diferencia; el índice único evita duplicar aplicación de una guía/producto.
LecturaGuia	Una fila por captura	Historial previo al cierre; conserva código, cantidad, lote, serie, ubicación, operador y fecha.

Consultas de stock

`StockRepository.ListarAsync` necesita `idAlmacen`; sin este filtro devuelve una lista vacía deliberadamente. Filtra ubicaciones activas y productos activos, carga unidad de medida y ordena por código/ubicación. `StockService` proyecta esos datos a DTO. `GET /api/stocks?idAlmacen=4&idProducto=23` es un ejemplo de consulta compacta para pantalla/reportes.

6. Seguridad, sesión y errores

Cookies distintas por tipo de cliente

`AuthCookieSchemes` define cookies separadas: `ProjectAW.Admin.Auth` y `ProjectAW.Operador.Auth`. Durante login, `AuthApiController` valida credenciales, evalúa origen y rol, crea claims (`NameIdentifier`, usuario, `IdRol`) y firma solo el esquema correspondiente. La cookie es HttpOnly y Secure en HTTPS. No se guarda token en localStorage o sessionStorage.

- La cookie HttpOnly reduce la exposición a JavaScript, pero no elimina XSS/CSRF ni sustituye controles de autorización.
- El selector de esquema usa el Origin configurado de la PWA para seleccionar Operador; un Origin propio del backend selecciona Admin; otros orígenes se rechazan.
- Las cookies pueden compartirse entre puertos del mismo host según reglas del navegador; nombres distintos hacen que el login de operador no reemplace la sesión de administrador.
- CORS lista orígenes permitidos y métodos/headers necesarios; evitar `AllowAnyOrigin` cuando se envían credenciales.
- En `LecturasApiController` se marca `[Authorize]`; además el servicio comprueba rol activo y propiedad de la guía. Autenticación, rol y pertenencia son chequeos distintos.

Limitación de intentos y logging

Program.cs define una ventana fija para login de 5 intentos por IP cada minuto, sin cola; excederla devuelve 429. `ApiExceptionHandler` registra el error completo en servidor con TraceId, mientras da una respuesta segura al navegador. En producción todavía se necesita protección perimetral, monitoreo, límites de tamaño, alertas y controles contra abuso a nivel de hosting/firewall: el limitador de login no es protección DDoS general.

Qué revisar en una llamada no autorizada

10. ¿La cookie correcta se emitió y el fetch usa `credentials: "include"`?
11. ¿La URL y el origen están en HTTPS y `Cors:AllowedOrigins` coincide exactamente?
12. ¿El rol coincide con la aplicación (Operador para PWA y rol de administración para web admin)?
13. ¿El servicio obtiene el usuario desde `ICurrentUserService` y no desde un campo enviado por el cliente?

Desarrollo futuro de permisos

Hoy la API global exige usuario autenticado, y el flujo de lectura también verifica rol Operador y asignación de guía. Para futuros módulos administrativos con roles más finos conviene introducir políticas o autorización por rol/claim en endpoints, además de reglas de propiedad en servicios. No basta con ocultar tarjetas en HTML.

7. Funciones del cliente PWA

Cliente HTTP: api-client.js

Función	Uso
requestJson(path, options)	Privada. Une base URL + ruta, envía credenciales, negocia JSON/Problem Details y normaliza errores como ApiError.
getJson(path)	GET JSON para listados, catálogos, detalle e historial.
postJson(path, body)	POST con JSON para login y registrar lectura.
post(path)	POST sin cuerpo para tomar, cerrar sesión y finalizar.
checkApiReachability(timeoutMs)	Prueba si el host API responde por HTTPS con timeout; no comprueba login ni endpoint de negocio.

Funciones compartidas para guías: guias.js

Función	Cuándo usarla
getGuides(tipo, estado)	Listar guías por operación/estado y ordenar por fecha programada.
getGuideById(id)	Volver a comprobar el detalle antes de pedir que se tome.
takeGuide(id)	Asignar una guía pendiente a operador autenticado.
getMyGuides(estado)	Consultar Mis guías con filtro opcional por estado.
getMyGuideById(id)	Detalle seguro de una guía asignada.
registerGuideReading(id, request)	Guardar una captura JSON individual.
getGuideReadingHistory(id)	Recuperar el historial persistido.
finalizeGuideReadings(id)	Pedir el cierre transaccional al backend.
getActiveLocations(almacen)	Cargar datalist de ubicaciones activas.
getReceptionLookups()	Enriquecer solo Recepción con nombres de proveedor y almacén.
getGuideDetailLookups(tipo, knownProducts?)	Enriquecer detalle/registro; permite reutilizar catálogo ya cargado.
showApiError(error), isUnauthorized(error)	Unificar mensajes y detectar expiración de sesión.
formatScheduledAt(value)	Dar formato local es-PE para fecha y hora.

Pantallas principales

Archivo	Función principal
inicio/home.js	Crea cards de Recepción/Despacho/Traslado, consulta conteos con Promise.allSettled y lleva a login ante 401.
guias/listado.js	Carga una operación, presenta tarjetas y confirma antes de tomar la guía.
guias/mis-guias.js	Tabs En proceso/Terminadas; permite reabrir detalle con el ID.
guias/detalle.js	Carga solo guías propias y ofrece Empezar registro si está en proceso.
guias/registro.js	Configura captura, catálogo, ubicaciones, historial, avance y finalización.

Archivo	Función principal
ajustes/ajustes.js	Guarda preferencia PDA/cámara en localStorage; no almacena credenciales.

Métodos de captura

PDA: el escáner se comporta como teclado, completa `#scan-code` y confirma Enter. `processScannedCode` restringe el código a los productos de esa guía. Cámara: `startCamera` pide permiso solo al activarla, `BarcodeDetector` lee formatos compatibles y `scanFrame` agenda otro intento cada 220 ms. `stopCamera` libera tracks al apagar, ocultar pestaña o abandonarla. La API del navegador puede no existir en ciertos dispositivos; se conserva entrada manual/PDA como alternativa.

8. Métodos menos frecuentes y utilidades

Estos métodos no son inútiles: su frecuencia depende de una pantalla, una fase del proceso o una condición puntual. El objetivo es saber cuándo modificarlos sin meter responsabilidades que no les corresponden.

Método / helper	Frecuencia y uso concreto	Por qué queda aislado
checkApiReachability	Al abrir login, al volver online y cada 30 s mientras login está visible.	Da un indicador de conectividad, no valida sesión. Un timeout con AbortController evita dejar la comprobación pendiente.
getReceptionLookups	Solo al listar Recepción.	Proveedor/almacén enriquecen tarjetas; despacho/traslado no necesitan esas consultas extra.
getGuideDetailLookups	En detalle de guía y captura.	Agrupa catálogos; `knownProducts` permite no volver a solicitar la lista de productos.
isUnauthorized	Se usa en el conteo paralelo de home.	En otras pantallas el catch necesita tratar otros errores junto con 401.
getGuideById	Antes del diálogo “¿tomar esta guía?”.	Evita confirmar con un dato viejo del listado.
formatScheduledAt	Varias pantallas; helper puro.	Mantiene un formato consistente y una salida alternativa si fecha es inválida.
ObtenerSiguienteNumeroAsync	Al crear guía.	Usa SEQUENCE SQL (ENT/SAL/TRA) con formato de seis dígitos; evita calcular MAX+1 bajo concurrencia.
Normalizar	Al persistir lectura, solo valores opcionales.	Convierte espacios vacíos a null y recorta texto; mantiene el modelo limpio.
SincronizarStockAlmacenAsync	Durante el cierre, después de mutar ubicaciones.	El resumen se deriva del detalle por ubicación; no hace falta recalcularlo al hacer GET.
ToGuiaOperadorDto / ToLecturaDto	En cada respuesta del flujo de operador.	Expone solo campos adecuados al cliente y no serializa navegación EF completa.
Resolver (AuthCookieSchemes)	Durante autenticación de una request.	Elige esquema de cookie según origen configurado; se centraliza para evitar reglas distintas por controlador.
Map(exception)	Solo cuando un request falla.	Traduce errores de dominio/SQL a códigos HTTP estables y mensajes seguros.

EF Core en una consulta

```
// Consulta para mostrar datos (no se van a editar):
var guia = await _context.CabeceraGuia.AsNoTracking()
    .Include(g => g.Detalles)
    .FirstOrDefaultAsync(g => g.IdGuia == id, ct);

// Actualización: entity tracked + SaveChangesAsync dentro del repositorio.
guia.IdEstadoGuia = EstadoGuiasIds.EnProceso;
await _context.SaveChangesAsync(ct);
```

Se usa AsNoTracking en listados e historial para evitar trabajo de tracking. En edición se necesita una instancia rastreada para que EF detecte cambios. Include se elige solo cuando se requieren datos relacionados, evitando cargar todo indiscriminadamente.

Métodos que conviene mantener privados

- `ValidarCabecera`, `ValidarDetalles`, `ValidarReferenciasAsync`: son pasos internos de un caso de uso; no necesitan convertirse en API aparte.
- `ToResponse`/`ToDto`: traducen a contratos para que la entidad no quede acoplada a JSON.
- `ValidarGuiaEnProceso`, `ValidarUbicacionesAsync`, `AjustarStockUbicacionAsync`: se reutilizan dentro de registrar/finalizar; privatizarlos impide llamarlos fuera de un flujo controlado.

- `clearFields`, `renderProgress`, `resolveLocationId`: son detalles de la pantalla de registro; guias.js solo debe exponer llamadas HTTP y utilidades compartidas.

9. Cómo extender el sistema con el patrón actual

Agregar un endpoint de negocio

14. Define el request/response DTO con DataAnnotations y límites coherentes con la columna SQL.
15. Agrega el método a la interfaz del servicio y del repositorio si requiere datos/persistencia.
16. Implementa validación de negocio y autorización en el servicio; no aceptes el ID de operador desde el JSON si sale de claims.
17. Implementa consulta EF con filtro por pertenencia/estado, `CancellationToken`, `AsNoTracking` cuando aplique, y transacción si la operación modifica varios saldos.
18. Expón una ruta REST descriptiva en el controller; devuelve el status code apropiado y documenta resumen XML.
19. Registra el servicio/repositorio en Program.cs y define política CORS si llama un navegador de otro origen.
20. Agrega el helper fetch en la PWA, conecta eventos de UI y muestra errores 401/403/409 sin duplicar reglas de negocio.
21. Si cambió esquema EF, crea y aplica migración del proyecto backend.

Ejemplo de uso PWA

```
// modules/operaciones/guias.js
export async function getMyGuideById(id) {
  return getJson(`api/simulacion/guias/mias/${encodeURIComponent(id)}`);
}

// modules/operaciones/guias/detalle.js
const guide = await getMyGuideById(guideId);
renderGuide(guide);
```

La pantalla consume el JSON; si necesita una nueva regla de acceso, el backend la implementa. La PWA no crea una llamada “a su propio controller”, ni la lógica se debe duplicar en ambos proyectos.

Ejemplo de migración

```
# Ejecutar desde la carpeta ProjectAW\ProjectAW
dotnet tool install --global dotnet-ef --version 10.0.12
dotnet ef migrations add NombreDescriptivo   --project .\ProjectAW.csproj   --startup-project
.\ProjectAW.csproj
dotnet ef database update   --project .\ProjectAW.csproj   --startup-project .\ProjectAW.csproj
```

¿Uno o dos proyectos? Las migraciones se crean y aplican con ProjectAW porque ahí viven AppDbContext y la conexión SQL. ProjectAW.PWA no tiene EF Core ni base propia. `--project` identifica dónde están DbContext/migraciones; `--startup-project` ejecuta la configuración que provee conexión/servicios; en esta solución ambos apuntan al mismo `.csproj`. Si dotnet-ef no se encuentra, debe instalarse como herramienta antes de ejecutar el comando.

Ejemplo de consulta desde línea de comandos HTTP

```
GET https://localhost:7054/api/guias?idTipoOperacionGuia=1&idEstadoGuia=1
Accept: application/json
Cookie: (la envía el navegador; no copiar credenciales en documentación)
```

Para probar manualmente, usar una cuenta de prueba y dejar que el login emita cookie. No incluir contraseñas/cookies reales en ejemplos, commits o capturas.

10. Catálogo del repositorio y glosario

Archivos que se consultan primero

Necesidad	Archivos para empezar
Cambiar rutas de guías	Modules/Guias/Controllers/GuiasApiController.cs; Modules/Guias/Services/IGuiaService.cs; Modules/Guias/Services/GuiaService.cs.
Cambiar captura/cierre	Modules/Simulacion/Controllers/LecturasApiController.cs; Modules/Stocks/Services/MovimientoStockService.cs; Modules/Stocks/Repositories/MovimientoStockRepository.cs.
Cambiar entidad/migración	Modules/Stocks/Entities/LecturaGuia.cs; LecturaGuiaConfiguration.cs; Infrastructure/Data/AppDbContext.cs; Migrations/.
Cambiar auth/CORS	Program.cs; Modules/Seguridad/Security/Controllers/AuthApiController.cs; HttpAuth/AuthCookieSchemes.cs; launchSettings.json.
Cambiar configuración API cliente	ProjectAW.PWA/wwwroot/config/api-config.js; shared/js/api-client.js.
Cambiar operación móvil	ProjectAW.PWA/wwwroot/modules/operaciones/guias.js y los módulos HTML/JS de inicio, listado, detalle, mis guías, registro y ajustes.
Cambiar estilos adaptables	ProjectAW.PWA/wwwroot/modules/operaciones/operaciones.css; css/site.css.

Glosario

Término	Significado en este sistema
API	Contrato HTTP/JSON que publica ProjectAW para el panel y clientes como la PWA.
DTO	Objeto de transferencia que define datos públicos de entrada o respuesta.
PWA	Interfaz web de operador con diseño adaptable; en el código actual se sirve como archivos estáticos.
Lectura	Captura individual persistida en historial. Aún no equivale a movimiento de stock.
Finalizar	Caso de uso transaccional que valida todas las capturas, mueve stock y cambia estado.
AsNoTracking	Consulta EF sin seguimiento de cambios, útil cuando la entidad solo se va a leer.
Claims	Datos firmados en la sesión, como ID de usuario/rol, utilizados por backend como identidad.
CORS	Política de navegador para permitir lectura de respuestas entre orígenes; no reemplaza autorización.
Migración	Cambio versionado de esquema de base de datos generado por EF Core desde el modelo.

Ideas para seguir mejorando

- Mantener OpenAPI/Swagger actualizado con ejemplos de requests y códigos de error; habilitarlo solo en entornos controlados.
- Agregar pruebas automatizadas de reglas críticas: toma concurrente, pertenencia de guía, rollback por falta de stock y cálculos de diferencias.
- Revisar métricas/logs con TraceId, límites por IP/usuario y protección perimetral antes de exponer la API públicamente.
- Conservar la regla arquitectónica: las pantallas muestran y capturan; el backend autoriza, valida y decide el resultado persistente.